[1] Tom Cargill, “Exception Handling: A False Sense of Security,” in *C++ Report* 6, no. 9 (November–December 1994). Also available at http://www.informit.com/content/images/020163371x/supplements/Exception_Handling_Article.html.

[2] Herb Sutter, *Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions* (Addison Wesley Professional, 1999).

For those of you who aren’t aware of the issue, consider a `stack<vector<int>>`. Now, a `vector` is a dynamically sized container, so when you copy a `vector` the library has to allocate some more memory from the heap in order to copy the contents. If the system is heavily loaded, or there are significant resource constraints, this memory allocation can fail, so the copy constructor for `vector` might throw a `std::bad_alloc` exception. This is especially likely if the `vector` contains a lot of elements. If the `pop()` function was defined to return the value popped, as well as remove it from the stack, you have a potential problem: the value being popped is returned to the caller only *after* the stack has been modified, but the process of copying the data to return to the caller might throw an exception. If this happens, the data just popped is lost; it has been removed from the stack, but the copy was unsuccessful! The designers of the `std::stack` interface helpfully split the operation in two: get the top element (`top()`) and then remove it from the stack (`pop()`), so that if you can’t safely copy the data, it stays on the stack. If the problem was lack of heap memory, maybe the application can free some memory and try again.

Unfortunately, it’s precisely this split that you’re trying to avoid in eliminating the race condition! Thankfully, there are alternatives, but they aren’t without cost.

Option 1: Pass in a Reference

The first option is to pass a reference to a variable in which you wish to receive the popped value as an argument in the call to `pop()`:

```
std::vector<int> result;
some_stack.pop(result);
```